Cómo correr SIRISYS en tu Mac

Guía paso a paso desde cero para la versión 13.3. No hace falta saber programar ni haber abierto nunca una Terminal.

Antes de empezar

Vas a necesitar:

- Tu Mac y conexión a internet.
- El repositorio **STRATA** de GitHub: **github.com/juanan-prz/STRATA** (el paso 1 explica cómo descargarlo).
- Para los experimentos con modelo: **Ollama**, gratis y local (recomendado), o una clave de API de Anthropic (opcional). Las pruebas de funcionamiento no necesitan ninguno de los dos.

La primera vez son unos 30 minutos de instalación. Después, cada sesión empieza en un minuto.

> **Regla de oro:** los comandos hay que copiarlos y pegarlos tal cual, sin cambiar nada. Un espacio de más o una tilde donde no toca y no funciona. Copiar y pegar es lo correcto, no hacer trampa.

Paso 1 · Descargar el proyecto

- Abre en Safari **https://github.com/juanan-prz/STRATA**
- Pulsa el botón verde **Code** y después **Download ZIP**.
- Abre el ZIP descargado (doble clic): aparece una carpeta.
- Muévela a **Documentos** y cámbiale el nombre a exactamente **sirisys** (minúsculas, sin acentos ni espacios).

Dentro deben estar, juntos y sin subcarpetas, estos archivos (entre otros):

Archivo	Para qué sirve
`Sirisys_Framework_v13_3.py`	El sistema.
`Sirisys_Server.py` · `sirisys_loader.py`	Arrancan el modo experimento, con el visualizador.
`run_report.py`	Ejecuciones medidas: genera el informe que me envías.
`field_topology.py` · `sirisys_measures.py`	Mediciones que usan los informes. Vigilan también que el modelo responda.
`test_*.py` · `sirisys_testkit.py`	Pruebas de funcionamiento: cinco archivos y lo que comparten.
`live_c2.py` · `soak.py`	Pruebas de resistencia: perceptos reales y vida larga.
`Sirisys_Live_Visualizer.html`	El visualizador en vivo.
`requirements.txt`	La lista de piezas que hay que instalar.

Descarga siempre desde el ordenador, no desde el móvil. Si los archivos pasan por el portapapeles del iPhone o por un editor como Kodex, se corrompen las comillas y el código deja de funcionar.

Cuando haya una versión nueva del framework, basta con dejarla en esta carpeta: el programa usa siempre la versión más alta y dice en pantalla cuál ha cargado.

Paso 2 · Abrir la Terminal

La Terminal es una ventana donde se escriben órdenes en vez de hacer clic. Pulsa **Command + barra espaciadora**, escribe **Terminal** y pulsa Enter. Se abre una ventana con un cursor parpadeando. No puedes romper nada escribiendo ahí: si un comando no existe, simplemente te lo dice.

Paso 3 · Comprobar si tienes Python

Copia esto en la Terminal y pulsa Enter:

```
python3 --version
```

Si responde **Python 3.10** o superior (por ejemplo 3.12.4), ya lo tienes: salta al paso 4. Si dice command not found, te ofrece instalar herramientas o la versión es más antigua: ve a **python.org/downloads**, pulsa el botón amarillo **Download Python**, instálalo como cualquier programa, cierra la Terminal, ábrela de nuevo y repite el comando.

Paso 4 · Situarte en la carpeta

La Terminal necesita saber dónde están los archivos. Escribe **cd** seguido de un espacio (sin pulsar Enter), arrastra la carpeta **sirisys** desde el Finder encima de la ventana de la Terminal y, cuando aparezca la ruta, pulsa Enter. Para comprobarlo:

```
ls
```

Debe responder con la lista de archivos del paso 1. Si los ves, vas bien.

Paso 5 · Instalar las piezas

```
pip3 install -r requirements.txt
```

Verás mucho texto pasando: es normal y tarda un par de minutos. Cuando vuelvas a poder escribir, ha terminado. Si sale un aviso en rojo que dice externally-managed-environment, usa esta otra línea:

```
pip3 install --break-system-packages -r requirements.txt
```

Paso 6 · Comprobar que todo funciona (gratis, sin modelo)

Lanza estas cinco pruebas, una a una. No gastan dinero ni necesitan internet:

Comando	Qué debe decir al final
`python3 test_sirisys.py`	SUMMARY: 24/24 passed (≈ 15 s)
`python3 test_phase1.py`	SUMMARY: 30/30 passed (≈ 15 s)
`python3 test_phase2_guarantee.py`	SUMMARY: 6/6 passed (≈ 5 s)
`python3 test_phase4.py`	SUMMARY: 12/12 passed (≈ 5 s)
`python3 test_ontology.py`	SUMMARY: 15/15 passed (≈ 5 s)

Si alguna línea dice **FAIL**, hazle una foto y mándamela antes de seguir. Para ver el campo en vivo:

```
python3 Sirisys_Server.py --play
```

Abre en Safari **http://127.0.0.1:8000**. Para pararlo, vuelve a la Terminal y pulsa **Control + C**. Cada arranque se guarda en la carpeta **sirisys_runs**; no se borra nada.

Paso 7 · Elegir el modelo

Opción A — Ollama (recomendada). El modelo corre dentro de tu Mac: gratis, sin enviar nada fuera.

- Descárgalo de **ollama.com**, instálalo como cualquier programa y ábrelo (queda un icono en la barra de arriba).
- En la Terminal, descarga un modelo. Por ejemplo:

```
ollama pull llama3.1
ollama list
```

ollama list debe mostrar el modelo descargado. Usa siempre el mismo modelo en las ejecuciones que quieras comparar.

Opción B — Anthropic (opcional, solo para experimentos). En **console.anthropic.com** crea una cuenta, ve a **API Keys → Create Key**, copia la clave (empieza por sk-ant-; solo se muestra una vez) y añade saldo en **Billing** sin activar la recarga automática. La suscripción a Claude no da saldo de API: son cosas separadas. Después, en la Terminal:

```
export ANTHROPIC_API_KEY="sk-ant-aqui-tu-clave"
```

No responde nada: eso significa que ha ido bien. Dura mientras la ventana esté abierta.

Paso 8 · La prueba corta del informe

```
python3 run_report.py --cycles 20 --seeds 1 --backend ollama --profile minimal
```

(Con Anthropic, cambia **ollama** por **anthropic**.) Antes de empezar, el programa hace dos preguntas reales al modelo. Fíjate en:

Qué buscar	Qué debe decir
Las 14 líneas de comprobación del principio	Todas con [ok].
model probe	ok, y los segundos que tarda cada llamada. Si el modelo no responde o responde vacío, el programa se para aquí con **STOP** y dice por qué (mira «Si algo falla»).
sample names	Dos nombres inventados por el modelo. Si suenan a tu terminología, va bien.
Al final: model calls	failures 0 y empty 0.

Paso 9 · Los experimentos que me envías

Todos con el perfil **minimal** (sin narración ni hora del día), en este orden. Delante de los largos va **caffeinate -i**: impide que el Mac se duerma mientras dura la prueba y deja de actuar sola al terminar. No cierres la ventana de la Terminal; en un portátil, déjalo enchufado y con la tapa abierta.

1 · Perceptos reales, sin modelo (≈ 10–15 minutos). La referencia: el campo con los perceptos de tu Mac durante una vida larga. **No uses el Mac mientras corre**: el campo percibe su procesador y su memoria.

```
python3 live_c2.py 10000 none minimal > live_none.json
```

2 · El modelo frente a un control sin modelo (varias horas). Tres brazos con los mismos tres flujos de perceptos: el modelo, el modelo que solo hace las preguntas al Sombunal, y el control. Al empezar dice qué modelo responde y cuántas horas tardará como máximo. Aquí sí puedes usar el Mac.

```
caffeinate -i python3 run_report.py --backend ollama --profile minimal --cycles 1000 --seeds 3
```

3 · Perceptos reales con el modelo (varias horas; buena para dejarla por la noche). Al empezar dice en la Terminal cuántas horas tardará como máximo. **No uses el Mac mientras corre.**

```
caffeinate -i python3 live_c2.py 3000 ollama minimal > live_ollama.json
```

4 · Opcional, recomendada: ¿el modelo responde siempre igual? (algo más de la mitad que la prueba 2). Repite dos veces la misma ejecución con el modelo y comprueba si el campo sale idéntico. Al terminar dice **determinism: YES** o **NO**: las dos respuestas sirven, no es un error. Escribe su propio archivo, así que no borra el de la prueba 2. Aquí sí puedes usar el Mac.

```
caffeinate -i python3 run_report.py --backend ollama --profile minimal --cycles 1000 --determinism
```

Mientras corre **live_c2.py** no se ve nada más en pantalla: es normal, escribe el resultado al terminar. **run_report.py** imprime una línea de progreso cada 25 ciclos.

> Si el modelo deja de responder a mitad (Ollama se cierra o se cuelga), el programa se detiene solo y lo dice en la Terminal con **STOPPED**. Abre Ollama y vuelve a lanzar ese mismo comando; el archivo interrumpido no hace falta.

Opcional: ver el campo en vivo con el modelo.

```
export SIRISYS_LLM_BACKEND=ollama
python3 Sirisys_Server.py --play --profile minimal
```

Abre **http://127.0.0.1:8000** como en el paso 6. Arriba se ve el modelo; si deja de responder, el campo se pausa, la cabecera lo marca en rojo (**watch tripped**) y el registro de eventos dice por qué.

Paso 10 · Enviarme los resultados

En la carpeta **sirisys** habrán aparecido **live_none.json**, **sirisys_report.json** y **live_ollama.json**, y **sirisys_determinism.json** si hiciste la 4. Adjúntalos al chat; si pesan demasiado, clic derecho → **Comprimir** y envía el .zip. Añade también:

- una foto de las cinco líneas SUMMARY del paso 6;
- el nombre del modelo de Ollama que usaste (lo dice **ollama list**);
- el chip y la memoria de tu Mac (menú Apple → **Acerca de este Mac**).

El tiempo de cada ejecución no hace falta medirlo: va dentro de los archivos. Si te atascas en cualquier paso, hazle una foto a la pantalla y mándamela.

Si algo falla

Lo que ves	Qué hacer
command not found: python3	Python no está instalado: vuelve al paso 3.
No such file or directory	No estás en la carpeta o faltan archivos: repite el paso 4 y comprueba con ls.
externally-managed-environment	Usa la segunda línea del paso 5.
Alguna prueba dice FAIL	Foto de la pantalla y mándamela. Si es un archivo corrupto, vuelve a descargar el ZIP desde el ordenador.
model active: False · STOP: … not available	Ollama no está abierto (ábrelo y repite), el modelo no está descargado (ollama list) o, con Anthropic, la clave no se puso en esta ventana.
STOP: … answered empty	El modelo «piensa» antes de responder y se queda sin espacio para el nombre. Usa un modelo general, por ejemplo llama3.1 (paso 7).
STOPPED — model health	El modelo dejó de responder a mitad. Abre Ollama y vuelve a lanzar ese comando.
authentication_error / 401 · credit balance is too low	Con Anthropic: la clave está mal copiada (cópiala entera, sin espacios) o falta saldo en console.anthropic.com → Billing.
already open by another writer	Ya hay otro SIRISYS escribiendo en esa carpeta: cierra la otra ventana de Terminal o pulsa Control + C en ella.

Lo que ves	Qué hacer
Address already in use	El visualizador ya está abierto en otra ventana: ciérrala con Control + C.
La prueba se cortó al dormirse el Mac	Vuelve a lanzarla con caffeinate -i delante; en un portátil, enchufado y con la tapa abierta.
La ventana parece parada	Normal: está trabajando. Si de verdad se colgó, Control + C.

Resumen: cada sesión

```
cd   (y arrastras aquí la carpeta sirisys)
python3 test_sirisys.py
python3 test_ontology.py
python3 live_c2.py 10000 none minimal > live_none.json
caffeinate -i python3 run_report.py --backend ollama --profile minimal --cycles 1000 --seeds 3
caffeinate -i python3 live_c2.py 3000 ollama minimal > live_ollama.json
caffeinate -i python3 run_report.py --backend ollama --profile minimal --cycles 1000 --determinism
```